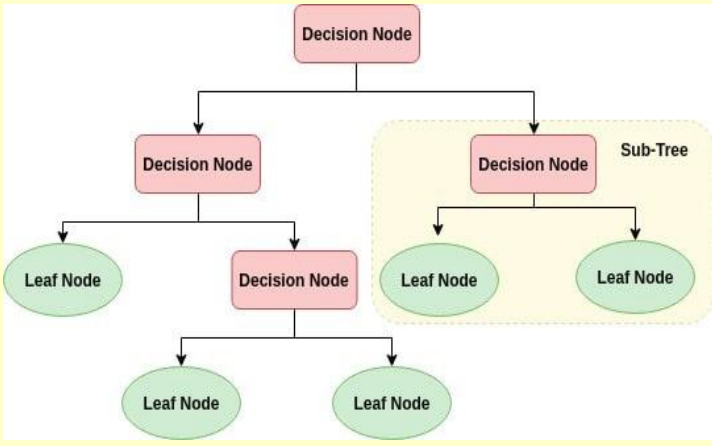
## Les arbres de décision

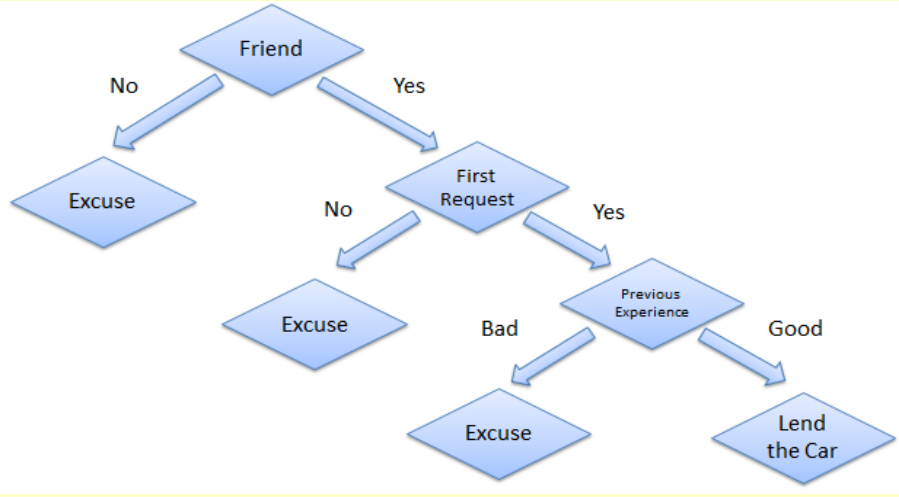
L'arbre de décision est un algorithme qui se base sur **un modèle d'arbre** pour définir la décision finale. Chaque nœud comporte une condition (une question), et les branchements sont en fonction de cette condition. Plus on descend dans l'arbre, plus on cumule les conditions.






## Les arbres de décision

Considérez un scénario dans lequel une personne vous demande de lui prêter votre voiture pour une journée, et vous devez décider de lui prêter ou non ta voiture.



## Les arbres de décision

L'utilisation des arbres de décision pour l'analyse prédictive présente plusieurs avantages:

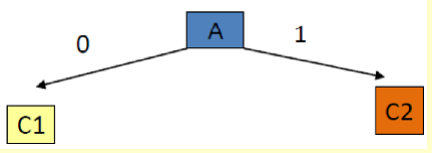
- Les arbres de décision peuvent être utilisés pour prédire des valeurs continues et discrètes, c'est-à-dire qu'ils fonctionnent bien pour les tâches de régression et de classification.
- Ils nécessitent relativement moins d'efforts pour former l'algorithme.
- Ils peuvent être utilisés pour classer des données séparables de manière non linéaire.
- Ils sont rapides et efficaces par rapport à KNN et à d'autres algorithmes de classification (revoir TP1-TP2).



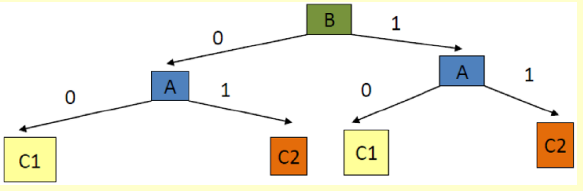
## Choix de l'attribut de partitionnement

| A | B | Classe |
|---|---|---|
| 0 | 1 | C1 |
| 0 | 0 | C1 |
| 1 | 1 | C2 |
| 1 | 0 | C2 |

Partitionnement selon l'attribut A




Partitionnement selon l'attribut B



## Choix de l'attribut de partitionnement

- Un arbre de décision représente la suite (≠ de ensemble) de questions à poser pour classer un nouveau exemple.
- Dans l'exemple précédent c'est l'attribut A qui apporte plus d'information à la classification, il est mieux que l'attribut B.

- **L'objectif est de minimiser le nombre de questions à poser.**

- **Question:** comment quantifier l'information apportée par chaque attribut ?



## Les algorithmes de construction d'un arbre de décision:

- ID3
- CART
- C4.5
- OC1
- SLIQ
- SPRINT

## Les algorithmes de construction d'un arbre de décision:

### L'entropy

- Une formule pour calculer l'homogénéité d'un échantillon.
- Un échantillon complètement homogène a une entropie de 0.
- La formule pour l'entropie est:

$$Entropie(S) = -1 * \sum_{c \in Classes} portion(c) * \log_2(portion(c))$$

Avec S est l'ensemble des instances (les lignes du data_train…)

## Choix de l'attribut de partitionnement

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.

Classes={C1,C2}

$$Entropie(S) = -1 * \sum_{c \in Classes(S)} portion(c) * \log_2(portion(c))$$

$$Entropie(S) = -1 * (0.5 * \log_2(0.5) + 0.5 * \log_2(0.5))$$

```
from math import *
>>> -1*(0.5*log2(0.5)+0.5*log2(0.5))
>>>1.0
```

98

## Choix de l'attribut de partitionnement

```python
from numpy import *
import pandas
df = pandas.DataFrame([[0,1,'C1'],[0,0,'C1'],
[1,1,'C2'],[1,0,'C2']],columns=['A', 'B', 'Classes'])
df=df.to_numpy()
```

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

```python
def Entropie(S):
    classes_distinctes=set(S[:,-1])
    classes=list(S[:,-1])
    s=0
    for c in classes_distinctes:
        p=classes.count(c)/len(classes)
        s+=p*log2(p)
    return(-1.0*s)
print(Entropie(df))
```

```
1.0
```

T15

99

## Les algorithmes de construction d'un arbre de décision: calcul du Gain d'information

- Il faut sélectionner l'attribut **avec le plus grand gain d'information**.
- Si on fait un partitionnement de l'ensemble d'instances **S** selon l'attribut ***Attribut*** nous obtenons des sous ensembles d'instances $\mathbf{S_i}$ avec i compris entre 1 et le nombre k de valeurs distincts de l'attribut ***Attribut:***

$$S = \bigcup_{i=1}^{k} S_i$$

- Il permet d'avoir l'attribut qui crée les branches les plus homogènes.
- Le gain d'information est basé sur la diminution de l'entropie après la division d'un ensemble de données par rapport à un attribut.
- ***Le Gain d'information est donné par la formule:***

$$Gain(S, Attribut) = Entropie(S) - \sum_{i=1}^{k} p_i * Entropie(S_i)$$

Avec pi est la portion du sous ensemble d'instances $\mathbf{S_i}$ par rapport à l'ensemble d'instances **S**:

$$pi = \frac{Nombre\ de\ lignes\ de\ S_i}{Nombre\ de\ lignes\ de\ S}$$

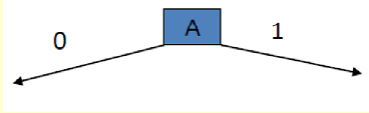
100

## Choix de l'attribut de partitionnement

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

**Partitionnement selon l'attribut A**




**S1** contient
{L1===C1
L2===C1}

**S2** contient
{L3===C2
L4===C2}

E(S1)=-1*(portion(C1)*log2(portion(C1)+ ~~portion(C2)*log2(portion(C2)))~~
=-1*(1*log2(1)) =0

Gain(S,A)=1-(portion(S1)*E(S1)+ portion(S2)*E(S2))
=1-((2/4)*E(S1) +(2/4)*E(S2))
=1- ((2/4)*(0) +(2/4)*(0))

**Gain(S,A)= 1**

101

## Choix de l'attribut de partitionnement

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

```python
def Gain_d_information(S,numero_colon_attribut):
    classes=list(set(S[:,-1]))
    valeurs_attribut=list(set(S[:,numero_colon_attribut]))
    Si=[[] for i in range(len(valeurs_attribut))]
    for e in S:
        e=list(e)
        val_attribut_pour_e=e[numero_colon_attribut]
        numero_sous_ensemble=valeurs_attribut.index(val_attribut_pour_e)
        Si[numero_sous_ensemble].append(e)
    Si=array(Si)  ;   som=0
    for sous_ensemble in Si:
        som+=(len(sous_ensemble)/len(S))*Entropie(sous_ensemble)
    return(Entropie(S)-som)
```

```python
print(Gain_d_information(df,0))
```

```
1.0
```

T15

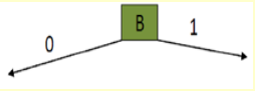
102

## Choix de l'attribut de partitionnement

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

**Partitionnement selon l'attribut B**




S1
{L2===C1
L4===C2}

S2
{L1===C1
L3===C2}

E(S1)=-1*(portion(C1)*log2(portion(C1)+ portion(C2)*log2(portion(C2)))
=-1*(0.5*log2(0.5)+0.5*log2(0,5)) =1.0
E(S2)=-1*(portion(C1)*log2(portion(C1)+ portion(C2)*log2(portion(C2)))
=-1*(0.5*log2(0.5)+0.5*log2(0,5)) =1.0

Gain(S,B)=1-(portion(S1)*E(S1)+ portion(S2)*E(S2))
=1-((2/4)*E(S1) +(2/4)*E(S2)) = 1- ((2/4)*(1.0) +(2/4)*(1.0)) = 0

**Gain(S,B)=0< Gain(S,A)=1 donc on doit faire le partitionnement selon A qui donne le plus grand gain.**



## Choix de l'attribut de partitionnement

```python
def Gain_d_information(S,numero_colon_attribut):
    classes=list(set(S[:,-1]))
    valeurs_attribut=list(set(S[:,numero_colon_attribut]))
    Si=[[] for i in range(len(valeurs_attribut))]
    for e in S:
        e=list(e)
        val_attribut_pour_e=e[numero_colon_attribut]
        numero_sous_ensemble=valeurs_attribut.index(val_attribut_pour_e)
        Si[numero_sous_ensemble].append(e)
    Si=array(Si)  ;   som=0
    for sous_ensemble in Si:
        som+=(len(sous_ensemble)/len(S))*Entropie(sous_ensemble)
    return(Entropie(S)-som)
```

```python
print(Gain_d_information(df,1))
```

```
0.0
```

T15

## Choix de l'attribut de partitionnement

| | A | B | C |
|---|---|---|---|
| 1 | Douleur | Inanime | Infarctus |
| 2 | poitrine | oui | oui |
| 3 | ailleurs | oui | oui |
| 4 | poitrine | non | oui |
| 5 | poitrine | oui | oui |
| 6 | ailleurs | non | oui |
| 7 | poitrine | non | non |
| 8 | ailleurs | non | non |
| 9 | poitrine | oui | non |
| 10 | ailleurs | non | non |
| 11 | ailleurs | non | non |

La répartition sera faite alors avec l'attribut 1 (colonne infarctus) qui donne le plus grand gain d'information

```
from numpy import *
import pandas
fich=pandas.read_csv('D:\\donneesAD1.csv',sep=';')
Data_train=fich.iloc[:,:]
Data_train=Data_train.to_numpy()
print(Gain_d_information(Data_train,0))
print(Gain_d_information(Data_train,1))
```

```
0.02904940554533142
0.12451124978365313
```

T16

105

## Choix de l'attribut de partitionnement: **Indice de GINI**

- Si on fait un partitionnement de l'ensemble d'instances **S** selon l'attribut ***Attribut*** nous obtenons des sous ensembles d'instances $\mathbf{S_i}$ avec i compris entre 1 et le nombre k de valeurs distincts de l'attribut ***Attribut:***

$$S = \bigcup_{i=1}^{k} S_i$$

- ***L'indice de GINI*** qui mesure le désordre est donné par la formule:

$$GINI(S) = 1 - \sum_{c \in Classes} (portion(c))^2$$

- **On prend l'attribut qui donne la plus petite mesure de désordre:**

$$Mesure_de_Désordre(S, Attribut) = \sum_{i=1}^{k} p_i * GINI(S_i)$$

Avec pi est la portion du sous ensemble d'instances $\mathbf{S_i}$ par rapport à l'ensemble d'instances **S**:

$$pi = \frac{Nombre\ de\ lignes\ de\ S_i}{Nombre\ de\ lignes\ de\ S}$$

106

## Choix de l'attribut de partitionnement: **Indice de GINI**

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

$$GINI(S) = 1 - \sum_{i=1}^{k} (p_i)^2$$

GINI(S)=1-((portion(C1))$^2$+ ((portion(C1))$^2$)
=1-((0.5)$^2$ +(0.5)$^2$)
=0.5



## Choix de l'attribut de partitionnement: **Indice de GINI**

```
def GINI(S):
    clonne_des_classes=list(S[:,-1])
    classes=list(set(S[:,-1]))
    k=len(classes) ;   s=0
    for i in range(k):
        pi=clonne_des_classes.count(classes[i])/len(S[:,-1])
        s+=pi**2
    return 1-s
```

```
from numpy import *
import pandas
df = pandas.DataFrame([[0,1,'C1'],[0,0,'C1'],
[1,1,'C2'],[1,0,'C2']],columns=['A', 'B', 'Classes'])
df=df.to_numpy()
print(GINI(df))
```

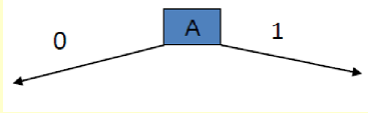
```
0.5
```

## Choix de l'attribut de partitionnement: **Indice de GINI**

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

**Partitionnement selon l'attribut A**




**S1** contient
{L1===C1
L2===C1}

**S2** contient
{L3===C2
L4===C2}

GINI(S1)=1-((portion(C1)*(portion(C1)) =0

GINI(S2)=1-((portion(C2)*(portion(C2)) =0

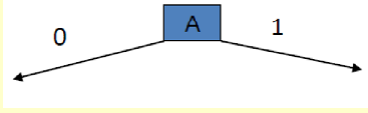
$$GINI(S) = 1 - \sum_{i=1}^{k} (p_i)^2$$



## Choix de l'attribut de partitionnement: **Indice de GINI**

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

**Partitionnement selon l'attribut A**




**S1** contient
{L1===C1
L2===C1}

**S2** contient
{L3===C2
L4===C2}

GINI(S1)=1-((portion(C1)*(portion(C1)) =0

GINI(S2)=1-((portion(C2)*(portion(C2)) =0

$$Mesure_de_Désordre(S, Attribut) = \sum_{i=1}^{k} p_i * GINI(S_i)$$

Mesure_de_desordre(S,A)=0.5*0+0.5*0=0

## Choix de l'attribut de partitionnement: **Indice de GINI**

```python
def Mesure_desordre(S,numero_colon_attribut):
    classes=list(set(S[:,-1]))
    valeurs_attribut=list(set(S[:,numero_colon_attribut]))
    Si=[[] for i in range(len(valeurs_attribut))]
    for e in S:
        val_attribut_pour_e=e[numero_colon_attribut]
        numero_sous_ensemble=valeurs_attribut.index(val_attribut_pour_e)
        Si[numero_sous_ensemble].append(e)
    Si=array(Si)
    s=0
    for sous_ensemble in Si:
        s+=(len(sous_ensemble)/len(S))*GINI(sous_ensemble)
    return(s)
```

```python
from numpy import *
import pandas
df = pandas.DataFrame([[0,1,'C1'],[0,0,'C1'],
[1,1,'C2'],[1,0,'C2']],columns=['A', 'B', 'Classes'])
df=df.to_numpy()
A=Mesure_desordre(df,0);print(A)
```

```
0.0
```

T17

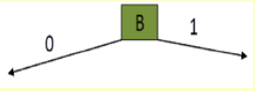


## Choix de l'attribut de partitionnement: **Indice de GINI**

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

**Partitionnement selon l'attribut B**




S1
{L2===C1
L4===C2}

S2
{L1===C1
L3===C2}

$$GINI(S) = 1 - \sum_{i=1}^{k} (p_i)^2$$

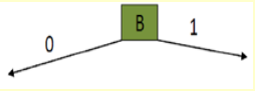
GINI(S1)=1-((portion(C1)*(portion(C1) +(portion(C2)*(portion(C2)) = 1-($0.5^2$+$0.5^2$) = 0.5
GINI(S2)= 1-((portion(C1)*(portion(C1) +(portion(C2)*(portion(C2)) = 1-($0.5^2$+$0.5^2$) = 0.5

## Choix de l'attribut de partitionnement: **Indice de GINI**

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

| Numéro de ligne | A | B | Classe |
|---|---|---|---|
| L1 | 0 | 1 | C1 |
| L2 | 0 | 0 | C1 |
| L3 | 1 | 1 | C2 |
| L4 | 1 | 0 | C2 |

**Partitionnement selon l'attribut B**




S1
{L2===C1
L4===C2}

S2
{L1===C1
L3===C2}

GINI(S1)=1-((portion(C1)*(portion(C1) +(portion(C2)*(portion(C2)) = 1-($0.5^2$+$0.5^2$) = 0.5
GINI(S1)= 1-((portion(C1)*(portion(C1) +(portion(C2)*(portion(C2)) = 1-($0.5^2$+$0.5^2$) = 0.5

$$Mesure_de_Désordre(S, Attribut) = \sum_{i=1}^{k} p_i * GINI(S_i)$$

Mesure_de_desordre(S,B)=0.5*0.5+0.5*0.5=0.5

**Mesure_de_desordre(S,B)> Mesure_de_desordre(S,A) donc on doit faire le partitionnement selon A qui donne la plus petite mesure de désordre.**



## Choix de l'attribut de partitionnement: **Indice de GINI**

```python
def Mesure_desordre(S,numero_colon_attribut):
    classes=list(set(S[:,-1]))
    valeurs_attribut=list(set(S[:,numero_colon_attribut]))
    Si=[[] for i in range(len(valeurs_attribut))]
    for e in S:
        val_attribut_pour_e=e[numero_colon_attribut]
        numero_sous_ensemble=valeurs_attribut.index(val_attribut_pour_e)
        Si[numero_sous_ensemble].append(e)
    Si=array(Si)
    s=0
    for sous_ensemble in Si:
        s+=(len(sous_ensemble)/len(S))*GINI(sous_ensemble)
    return(s)
```

```python
from numpy import *
import pandas
df = pandas.DataFrame([[0,1,'C1'],[0,0,'C1'],
[1,1,'C2'],[1,0,'C2']],columns=['A', 'B', 'Classes'])
df=df.to_numpy()
A=Mesure_desordre(df,1);print(A)
```

```
0.5
```

## Choix de l'attribut de partitionnement: **Indice de GINI**

| | A | B | C |
|---|---|---|---|
| 1 | Douleur | Inanime | Infarctus |
| 2 | poitrine | oui | oui |
| 3 | ailleurs | oui | oui |
| 4 | poitrine | non | oui |
| 5 | poitrine | oui | oui |
| 6 | ailleurs | non | oui |
| 7 | poitrine | non | non |
| 8 | ailleurs | non | non |
| 9 | poitrine | oui | non |
| 10 | ailleurs | non | non |
| 11 | ailleurs | non | non |

La répartition sera faite alors avec l'attribut 1 (colonne infarctus) qui donne la plus petite mesure de désordre

```
from numpy import *
import pandas
fich=pandas.read_csv('D:\\donneesAD1.csv',sep=';')
Data_train=fich.iloc[:,:]
Data_train=Data_train.to_numpy()
print(Mesure_desordre(Data_train,0))
print(Mesure_desordre(Data_train,1))
```

```
0.48
0.4166666666666667
```

T18



## Choix de l'attribut de partitionnement: **Indice de GINI**

```
from numpy import *
import pandas
f = pandas.read_csv('D:\\balance-scale1.data')
print(f.shape[0])
Data_train=f.iloc[0:8,:]
Data_train=Data_train.to_numpy()

def GINI(S):
    S=array(S)
    clonne_des_classes=list(S[:,-1])
    classes=list(set(S[:,-1]))
    k=len(classes)
    s=0
    for i in range(k):
        pi=clonne_des_classes.count(classes[i])/len(S[:,-1])
        s+=pi**2
    return 1-s
```

T20

## Choix de l'attribut de partitionnement: **Indice de GINI**

- Dans les exemples précédents nous avons considéré des attributs catégoriels: Douleur(poitrine, ailleurs), inanime(oui, non) ….
- Attributs continues (age, poids, taux de réussite, ….)
- Ici nous considérons que les attributs A,B et C sont continus

| A,B,C,Y | |
|---|---|
| 1,1,2,oui | L1 |
| 1,4,6,oui | L2 |
| 1,1,3,oui | L3 |
| 1,5,7,non | L4 |
| 2,2,3,non | L5 |
| 2,6,5,non | L6 |
| 2,8,9,non | L7 |
| 2,3,3,non | L8 |

Si on prend un attribut Note_du_Bac (avec deux chiffres après la virgule), si on a un data set qui contient cet attribut. Combien de sous ensemble obtenez vous si vous faites un regroupement selon cet attribut ?

**Exemple:** Note_du_Bac Note_du_Bac=[8,9.2,9.6,11.5,13,14,15,17,16.6,18,19]
Gain d'information si on utilise la formule précédente ? Est-il maximal ?



## Choix de l'attribut de partitionnement: **Indice de GINI**

T20

```python
def Mesure_desordre(S,numero_colon_attribut):
    Bonne_mesure_desordre=+inf
    Bonne_valeur_de_repartition=0
    classes=list(set(S[:,-1]))
    valeurs_attribut=list(set(S[:,numero_colon_attribut]))
    valeurs_attribut.sort()
    Gauche=[]; Droite=[]
    for i in range(len(valeurs_attribut)-1):#le dernier est le max !
        v=valeurs_attribut[i]; Gauche.clear(); Droite.clear()
        for e in S:
            val_attribut_pour_e=e[numero_colon_attribut]
            if val_attribut_pour_e<=v:
                Gauche.append(e)
            else:
                Droite.append(e)
        pGau=len(Gauche)/len(S)
        pDr=len(Droite)/len(S)
        mesure=(pGau)*GINI(Gauche)+(pDr)*GINI(Droite)
        if mesure<Bonne_mesure_desordre:
            Bonne_mesure_desordre=mesure
            Bonne_valeur_de_repartition=v
    return(Bonne_mesure_desordre,Bonne_valeur_de_repartition)
```

| A,B,C,Y | |
|---|---|
| 1,1,2,oui | L1 |
| 1,4,6,oui | L2 |
| 1,1,3,oui | L3 |
| 1,5,7,non | L4 |
| 2,2,3,non | L5 |
| 2,6,5,non | L6 |
| 2,8,9,non | L7 |
| 2,3,3,non | L8 |

**Exemple**: numero_colonne=1(B) et valeur=4
Gauche=[L1,L2,L3,L5,L8] et Droite=[L4,L6,L7]

## Choix de l'attribut de partitionnement: **Indice de GINI**

```
print(Mesure_desordre(Data_train,0))
print(Mesure_desordre(Data_train,1))
print(Mesure_desordre(Data_train,2))
```

```
(0.1875, 1)
(0.20833333333333326, 1)
(0.35714285714285715, 2)
```

| | A,B,C,Y | |
|---|---|---|
| S1 | 1,1,2,oui | L1 |
| | 1,4,6,oui | L2 |
| | 1,1,3,oui | L3 |
| | 1,5,7,non | L4 |
| S2 | 2,2,3,non | L5 |
| | 2,6,5,non | L6 |
| | 2,8,9,non | L7 |
| | 2,3,3,non | L8 |

T20



## Choix de l'attribut de partitionnement: **Indice de GINI**

```
def meilleur_attribut_meilleure_valeur(S):
    meilleur_attribut=0
    meilleure_val=0
    meilleure_mesure=+inf
    for num_attribut in range(len(S[0])-1):
        mesure,valeur=Mesure_desordre(S,num_attribut)
        if mesure<meilleure_mesure:
            meilleure_val=valeur
            meilleure_mesure=mesure
            meilleur_attribut=num_attribut
    return meilleur_attribut,meilleure_val
print(meilleur_attribut_meilleure_valeur(Data_train))
```

```
(0, 1)
```

| | A,B,C,Y | |
|---|---|---|
| S1 | 1,1,2,oui | L1 |
| | 1,4,6,oui | L2 |
| | 1,1,3,oui | L3 |
| | 1,5,7,non | L4 |
| S2 | 2,2,3,non | L5 |
| | 2,6,5,non | L6 |
| | 2,8,9,non | L7 |
| | 2,3,3,non | L8 |

T20

## Matrice de confusion

**Définition**: **Matrice de confusion**

Soit un Data_test qui contient **m** échantillons (m exemples) dont les classes sont "Positif " ou " Négatif ". Avec un modèle de ML on classe ces m échantillons. Ces échantillons seront réparties en 4 groupes:

- Le premier groupe contient les Vp échantillons **+**(positifs) qui sont classés +
- Le deuxième contient les Fp échantillons **+** qui sont classés **–**
- Le troisième groupe contient les Vn échantillons **–** qui sont classés **–**
- Le quatrième groupe contient les Fn échantillons **–** qui sont classés **+**

**Matrice de confusion** :

| | Classé + | Classé - |
|---|---|---|
| Exemple + | $V_p$ | $F_n$ |
| Exemple - | $F_p$ | $V_n$ |

Taux d'erreur = 1 - taux de réussite(*accuracy*) = $\frac{F_p+F_n}{F_p+F_n+V_p+V_n}$

121

## Matrice de confusion

**Exemple**: **Matrice de confusion**

Supposons dans l'exemple de détection des SMS spam (faux sms) des SMS ham (vrai SMS). Le classificateur a été testé par exemple sur 200 SMS.

| | | Classe estimée - (par le classificateur) | |
|---|---|---|---|
| | | Ham | Spam |
| Classe réelle - (selon le destinataire humain des sms) | Ham | 95 (vrais positifs) | 5 (faux négatifs) |
| | Spam | 3 (faux positifs) | 97 (vrais négatifs) |

Taux d'erreur = 1 - taux de réussite(*accuracy*) = accuracy = $\frac{\square + \square}{\square + \square + \square + \square}$

122

## Les arbres de décision

```python
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import *
```

```python
data = pd.read_csv('D:\\balance-scale1.data').as_matrix()
# Printing the dataswet shape
print ("Dataset Length: ", len(data))
print ("Dataset Shape: ", data.shape)
print(data)
```

```
[[1 1 2 'oui']
 [1 4 6 'oui']
 [1 1 3 'oui']
 [1 5 7 'non']
 [2 2 3 'non']
 [2 6 5 'non']
 [2 8 9 'non']
 [2 3 3 'non']
 [2 6 9 'non']
 [2 8 9 'non']
 [1 8 7 'oui']
 [1 9 6 'oui']
 [1 0 4 'oui']]
```

T19

## Les arbres de décision

```python
X = data[:8,0:3]; Y = data[:8,3] ; X_test=data[8:,0:3];  y_test=data[8:,3]
clf_gini = DecisionTreeClassifier(criterion = "gini")
clf_gini.fit(X, Y)
clf_entropy = DecisionTreeClassifier(criterion = "entropy")
clf_entropy.fit(X, Y)
print("Results Using Gini Index:")
y_pred_gini = clf_gini.predict(X_test)
print("Confusion Matrix: ",confusion_matrix(y_test, y_pred_gini))
print(X_test,y_test,y_pred_gini)
print ("Accuracy : ",accuracy_score(y_test,y_pred_gini)*100)
print("Results Using Entropy:")
y_pred_entropy = clf_entropy.predict(X_test)
print("Confusion Matrix: ",confusion_matrix(y_test, y_pred_entropy))
print ("Accuracy : ",accuracy_score(y_test,y_pred_entropy)*100)
```

T19

## Les arbres de décision

```
Results Using Gini Index:
Confusion Matrix:
```

| | prédite non | prédite oui |
|---|---|---|
| Les non | [[2 | 0] |
| Les oui | [1 | 2]] |

Classes correctes données par l'expert ['non' 'non' 'oui' 'oui' 'oui']

Classes prédites par AD-Gini ['non' 'non' 'non' 'oui' 'oui']

```
Accuracy :  80.0
Results Using Entropy:
Confusion Matrix:  [[2 0]
                    [2 1]]
Accuracy :  60.0
```

Le nombre d'exemples qui sont bien classés est égal à la trace de la matrice de confusion: trace(MC)=somme de la diagonale (MC)

T19

125

## Les arbres de décision

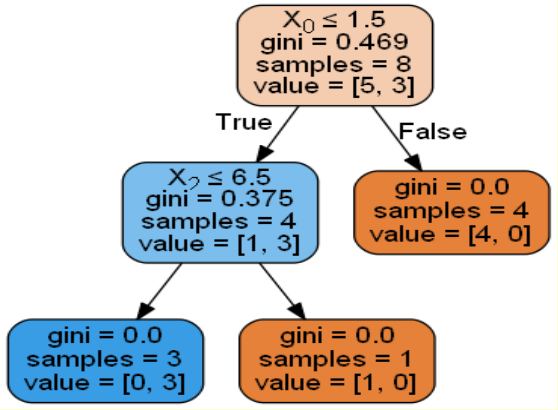
```python
from sklearn.externals.six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydotplus
```

```python
dot_data = StringIO()
export_graphviz(clf_gini, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())

svg_graph = graph.create_svg()
svg = open('gini_tree.svg', 'wb')
svg.write(svg_graph)
svg.close()
Image(graph.create_png())
```

T19

126

## Les arbres de décision

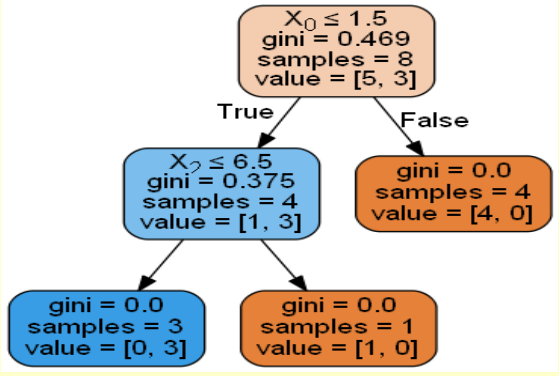



| A,B,C,Y | |
|---|---|
| 1,1,2,oui | L1 |
| 1,4,6,oui | L2 |
| 1,1,3,oui | L3 |
| 1,5,7,non | L4 |
| 2,2,3,non | L5 |
| 2,6,5,non | L6 |
| 2,8,9,non | L7 |
| 2,3,3,non | L8 |

Le data train S={L1,L2,L3,L4,L5,L6,L7,L8} contient 8 échantillons (samples=8)
5 échantillons ont la classe "non" et 3 échantillons ont la classe "oui" (value=[5,3])



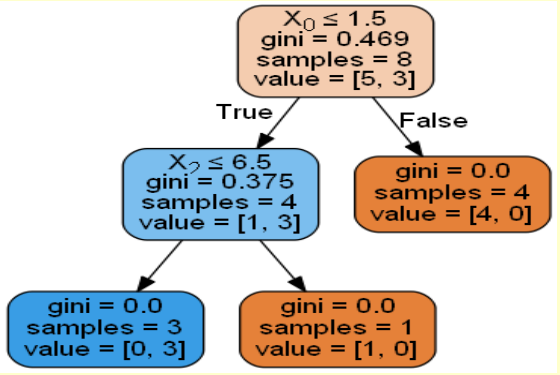
## Les arbres de décision

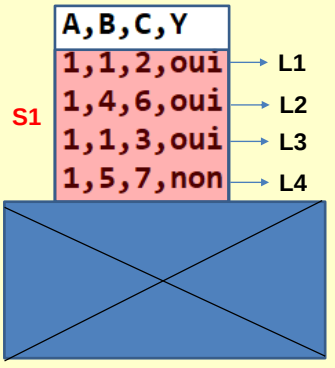



| | A,B,C,Y | |
|---|---|---|
| S1 | 1,1,2,oui | L1 |
| S1 | 1,4,6,oui | L2 |
| S1 | 1,1,3,oui | L3 |
| S1 | 1,5,7,non | L4 |
| S2 | 2,2,3,non | L5 |
| S2 | 2,6,5,non | L6 |
| S2 | 2,8,9,non | L7 |
| S2 | 2,3,3,non | L8 |

La condition $X_0 \leq 1.5$ (ici $X_0$ =colonne A) permet de diviser l'ensemble S={L1,L2,L3,L4,L5,L6,L7,L8} en deux ensembles de cardinal 4:
S1={L1,L2,L3,L4} et S2={L5,L6,L7,L8}
Les échantillons de S1 sont ceux qui vérifient la condition:
$X_0 \leq 1.5$ (le mot True), dans S1: 3 échantillons ont la classe "oui" et 1 échantillon a la classe "non", donc l'ensemble S1 n'est pas homogène !

## Les arbres de décision

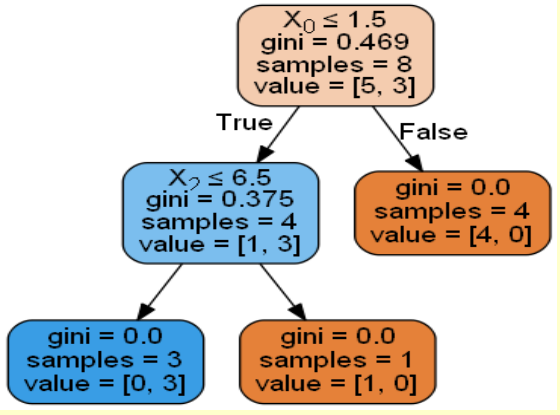


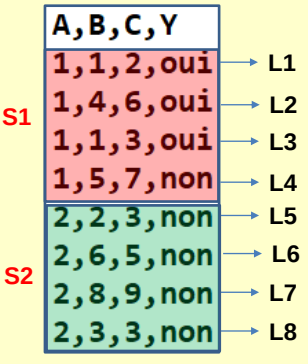




La condition $X_2 \leq 6.5$ (ici $X_2$ =colonne C) permet de diviser l'ensemble S1={L1,L2,L3,L4} en deux ensembles :
S3={L1,L2,L3} et S4={L4}
Les échantillons de S3 sont ceux qui vérifient la condition:$X_2 \leq 6.5$
dans S3, tous les échantillons ont la classe "oui" et S3 est homogène
Les échantillons de S4 sont ceux qui ne vérifient pas la condition:$X_2 \leq 6.5$
dans S4, tous les échantillons ont la classe "non" et S4 est homogène



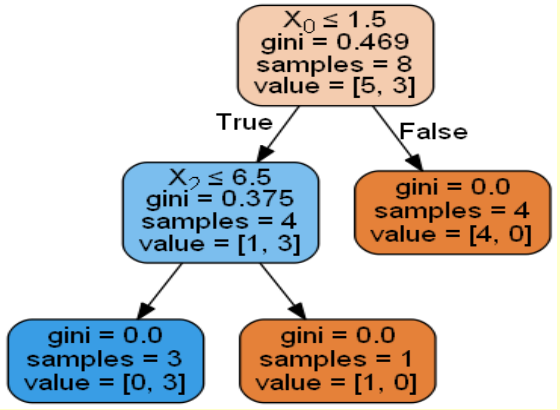
## Les arbres de décision




La condition $X_0 \leq 1.5$ (ici $X_0$ =colonne A) permet de diviser l'ensemble S={L1,L2,L3,L4,L5,L6,L7,L8} en deux ensembles de cardial 4:
S1={L1,L2,L3,L4} et S2={L5,L6,L7,L8}
Les échantillons de S2 sont ceux qui ne vérifient pas la condition:
$X_0 \leq 1.5$ (le mot False) et ils sont tous dans la classe "non"

## Les arbres de décision

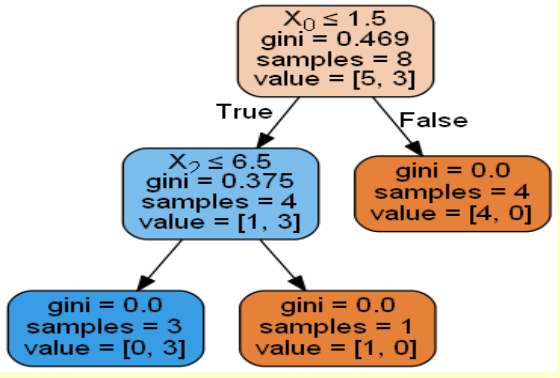



| | A,B,C,Y | |
|---|---|---|
| S1 | 1,1,2,oui | L1 |
| | 1,4,6,oui | L2 |
| | 1,1,3,oui | L3 |
| | 1,5,7,non | L4 |
| S2 | 2,2,3,non | L5 |
| | 2,6,5,non | L6 |
| | 2,8,9,non | L7 |
| | 2,3,3,non | L8 |

Les échantillons de S2 sont ceux qui ne vérifient pas la condition: $X_0 \leq 1.5$ (le mot False) et ils sont tous dans la classe "non" donc S2 est homogène,



## Les arbres de décision



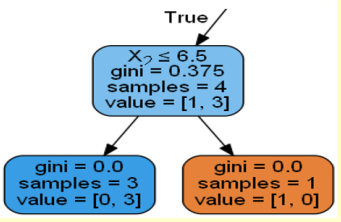

| | A,B,C,Y | |
|---|---|---|
| S1 | 1,1,2,oui | L1 |
| | 1,4,6,oui | L2 |
| | 1,1,3,oui | L3 |
| | 1,5,7,non | L4 |
| S2 | 2,2,3,non | L5 |
| | 2,6,5,non | L6 |
| | 2,8,9,non | L7 |
| | 2,3,3,non | L8 |

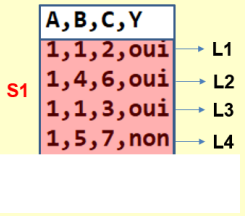
```python
fich=pandas.read_csv('D:\\balance-scale1.data')
Data_train=fich.iloc[:8,:]
Data_train=Data_train.to_numpy()
print(GINI(Data_train))
print(meilleur_attribut_meilleure_valeur(Data_train))
```

```
0.46875
(0, 1)
```

Tp 21

## Les arbres de décision



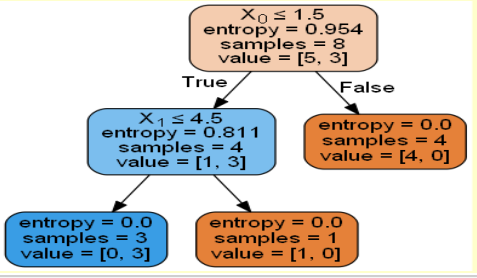

```
Data_train=array([[1, 1, 2, 'oui'],[1, 4, 6, 'oui'],
                  [1, 1, 3, 'oui'],[1, 5, 7, 'non']])
print(GINI(Data_train), end='  ')
print(Mesure_desordre(Data_train,0), end='  ')
print(Mesure_desordre(Data_train,1), end='  ')
print(Mesure_desordre(Data_train,2), end='  ')
print('--->'+str(meilleur_attribut_meilleure_valeur(Data_train)))
```

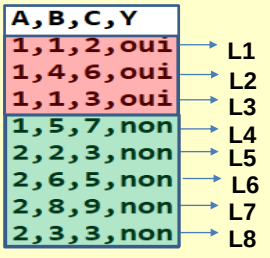
```
0.375  (inf, 0)  (0.0, '4')  (0.0, '6')  --->(1, '4')
```

Tp 21



## Les arbres de décision



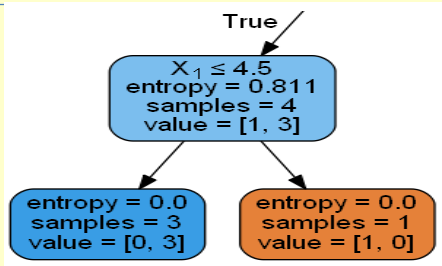



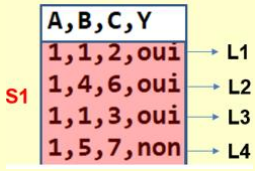

```
from numpy import *
import pandas
fich=pandas.read_csv('D:\\balance-scale1.data')
Data_train=fich.iloc[:8,:]
Data_train=Data_train.to_numpy()
print(Entropie(Data_train))
print(Mesure_Gain_d_information_continue(Data_train,0), end='  ')
print(Mesure_Gain_d_information_continue(Data_train,1), end='  ')
print(Mesure_Gain_d_information_continue(Data_train,2), end='  ')
```

Tp 22

```
0.954434002924965
(0.5487949406953987, 1)  (0.46691718668869947, 1)  (0.20443400292496505, 6)
```

## Les arbres de décision




```python
Data_train=array([[1, 1, 2, 'oui'],[1, 4, 6, 'oui'],
                  [1, 1, 3, 'oui'],[1, 5, 7, 'non']])
print(Entropie(Data_train))
print(Mesure_Gain_d_information_continue(Data_train,0), end='  ')
print(Mesure_Gain_d_information_continue(Data_train,1), end='  ')
print(Mesure_Gain_d_information_continue(Data_train,2), end='  ')

```

```
0.8112781244591328
(0, 0)  (0.8112781244591328, '4')  (0.8112781244591328, '6')
```

Tp 22



## Les arbres de décision

```python
def transformer_colText_Num(Data,num):
    if(type(Data[0][num])==str):
        Valeurs=list(set(Data[:,num]))
        for i in range(len(Data[:,num])):
            Data[i][num]=Valeurs.index(Data[i][num])
```

```python
data = pd.read_csv('D:\\donneesAD.csv',sep=';').as_matrix()
print(data)
for i in range(data.shape[1]-1):
    transformer_colText_Num(data,i)
print(data)
```

| Douleur | Age | Inanime | Infarctus |
|---|---|---|---|
| poitrine | 45 | oui | oui |
| ailleurs | 25 | oui | oui |
| poitrine | 35 | non | oui |
| Tete | 70 | oui | oui |
| ailleurs | 34 | non | oui |
| poitrine | 60 | non | non |
| ailleurs | 67 | non | non |
| Tete | 52 | oui | non |
| ailleurs | 58 | non | non |
| ailleurs | 34 | non | non |

```
[[0 45 1 'oui']
 [2 25 1 'oui']
 [0 35 0 'oui']
 [1 70 1 'oui']
 [2 34 0 'oui']
 [0 60 0 'non']
 [2 67 0 'non']
 [1 52 1 'non']
 [2 58 0 'non']
 [2 34 0 'non']]
```

Tp 23

## Les arbres de décision

```
X = data[:8,0:3]; Y = data[:8,3] ; X_test=data[8:,0:3];  y_test=data[8:,3]
clf_gini = DecisionTreeClassifier(criterion = "gini")
clf_gini.fit(X, Y)
print("Results Using Gini Index:")
y_pred_gini = clf_gini.predict(X_test)
print("Confusion Matrix: ",
      confusion_matrix(y_test, y_pred_gini))
print ("Accuracy : ",
        accuracy_score(y_test,y_pred_gini)*100)
```

```
Results Using Gini Index:
Confusion Matrix:  [[1 1]
 [0 0]]
Accuracy :  50.0
```

```
from sklearn.externals.six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydotplus
```

Tp 23

137

## Les arbres de décision

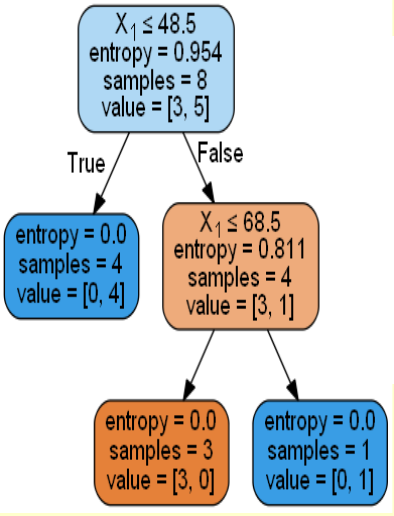
```
from sklearn.externals.six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydotplus
```

```
dot_data = StringIO()
export_graphviz(clf_gini, out_file=dot_data,
                filled=True, rounded=True,
                special_characters=True)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())

svg_graph = graph.create_svg()
svg = open('gini_tree.svg', 'wb')
svg.write(svg_graph)
svg.close()

Image(graph.create_png())
```

Tp 23

138

## Les arbres de décision




```
[['poitrine' 45 'oui' 'oui']——[[1 45 1 'oui']
 ['ailleurs' 25 'oui' 'oui']—— [2 25 1 'oui']
 ['poitrine' 35 'non' 'oui']—— [1 35 0 'oui']
 ['Tete' 70 'oui' 'oui']   —— [0 70 1 'oui']
 ['ailleurs' 34 'non' 'oui']—— [2 34 0 'oui']
 ['poitrine' 60 'non' 'non']—— [1 60 0 'non']
 ['ailleurs' 67 'non' 'non']—— [2 67 0 'non']
 ['Tete' 52 'oui' 'non']  ]—— [0 52 1 'non']]
```

Tp 23